

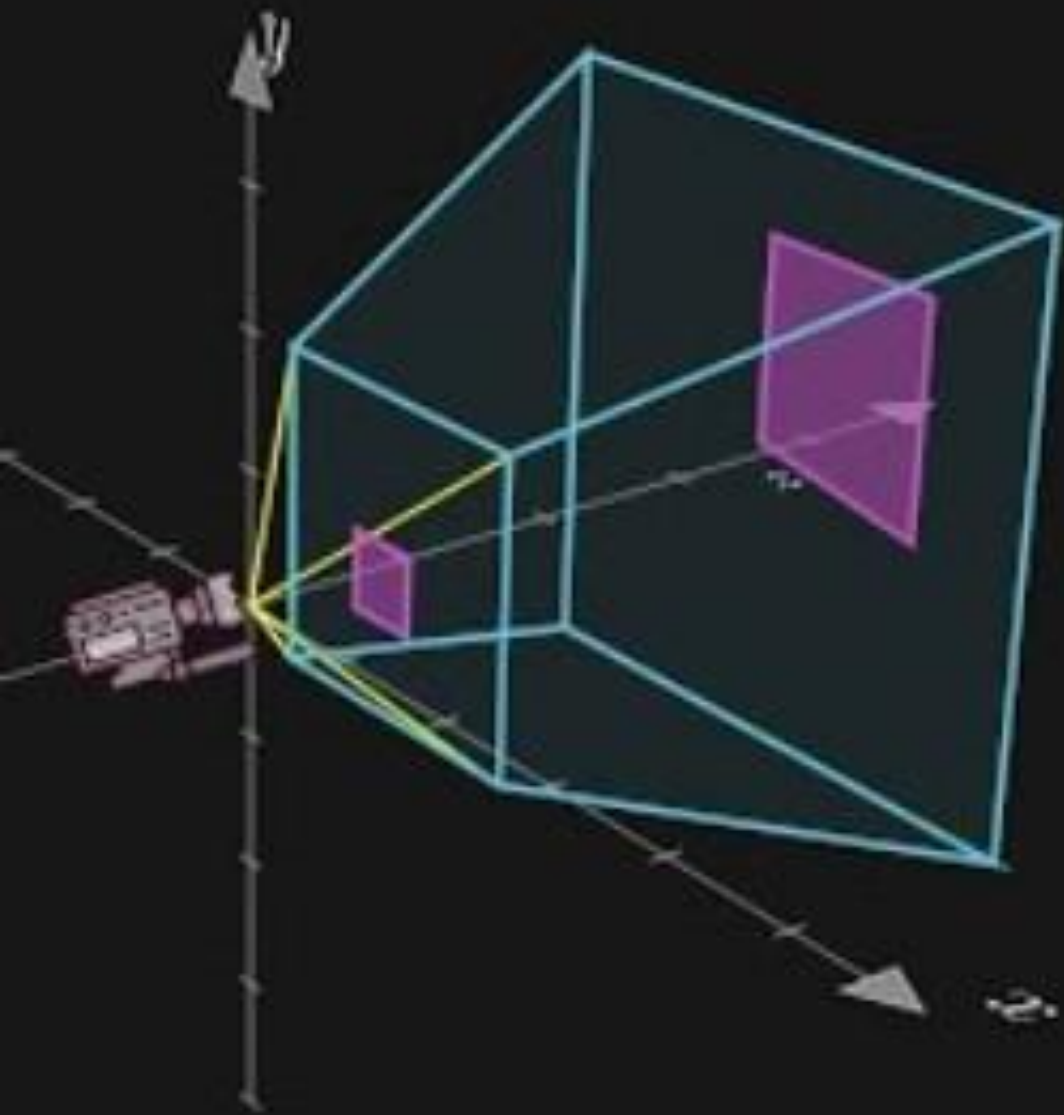
Transforms in Graphics Pipeline

DongJoon Kim

| Phone : +82-2-940-8461

| Email : dongjoonkim@kw.ac.kr

Review: Homogenous Coordinate System

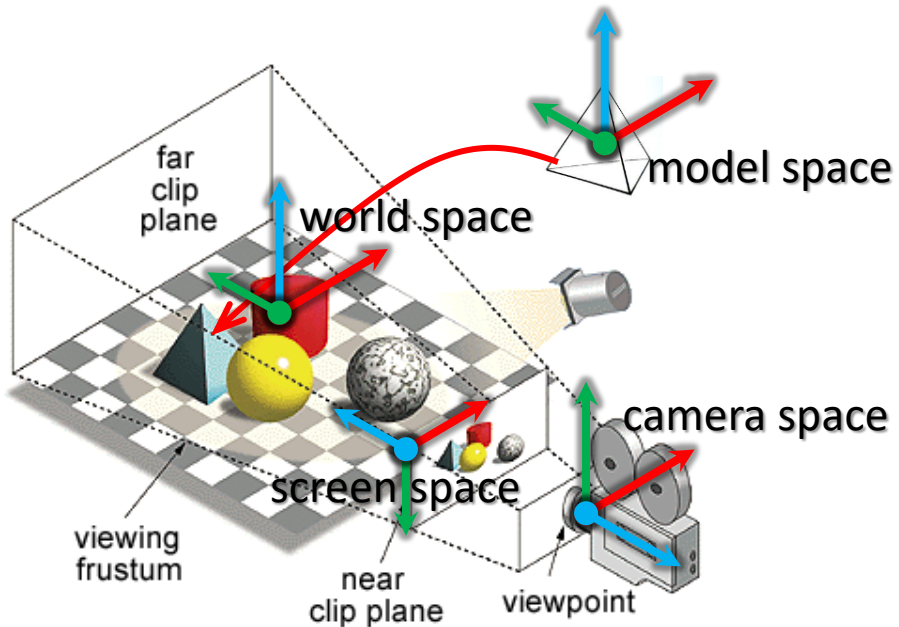
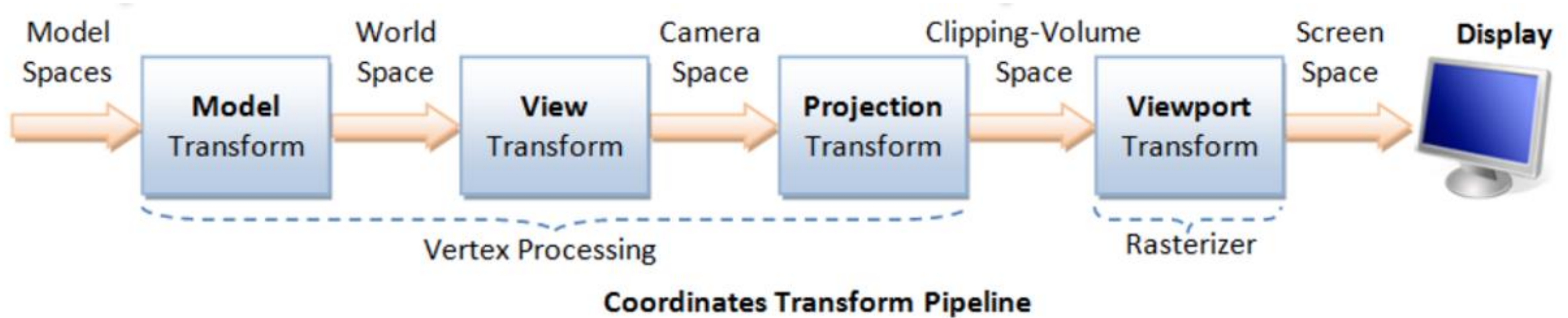


$$\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}$$

4D
vector

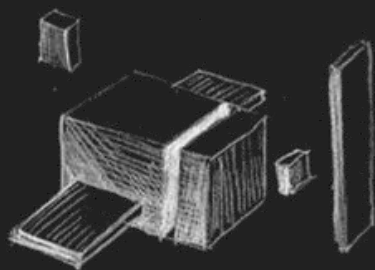
3D
object

렌더링에서의 변환 파이프라인

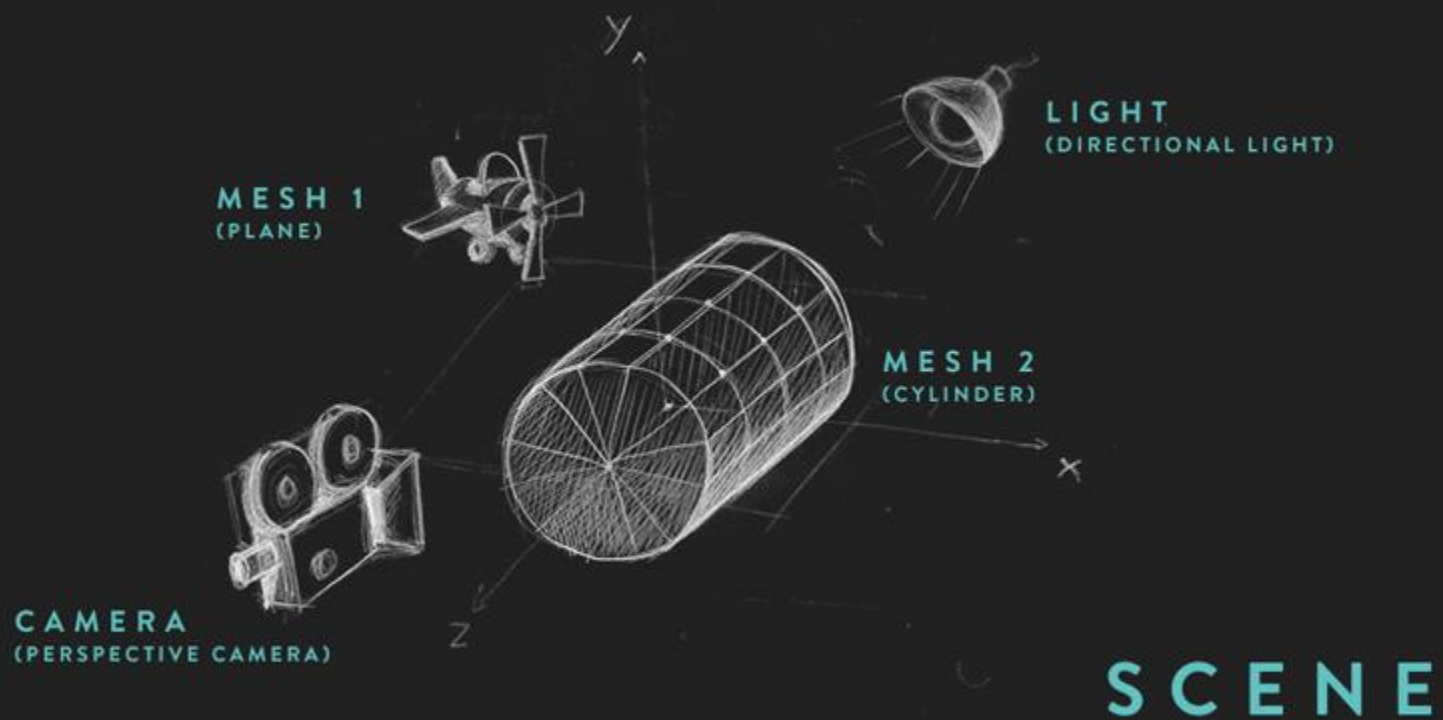


**** 여러 변환 행렬 (4×4) 들의 조합은
하나의 변환 행렬로 표현됨 (하나의 변환)**

Rendering Transforms

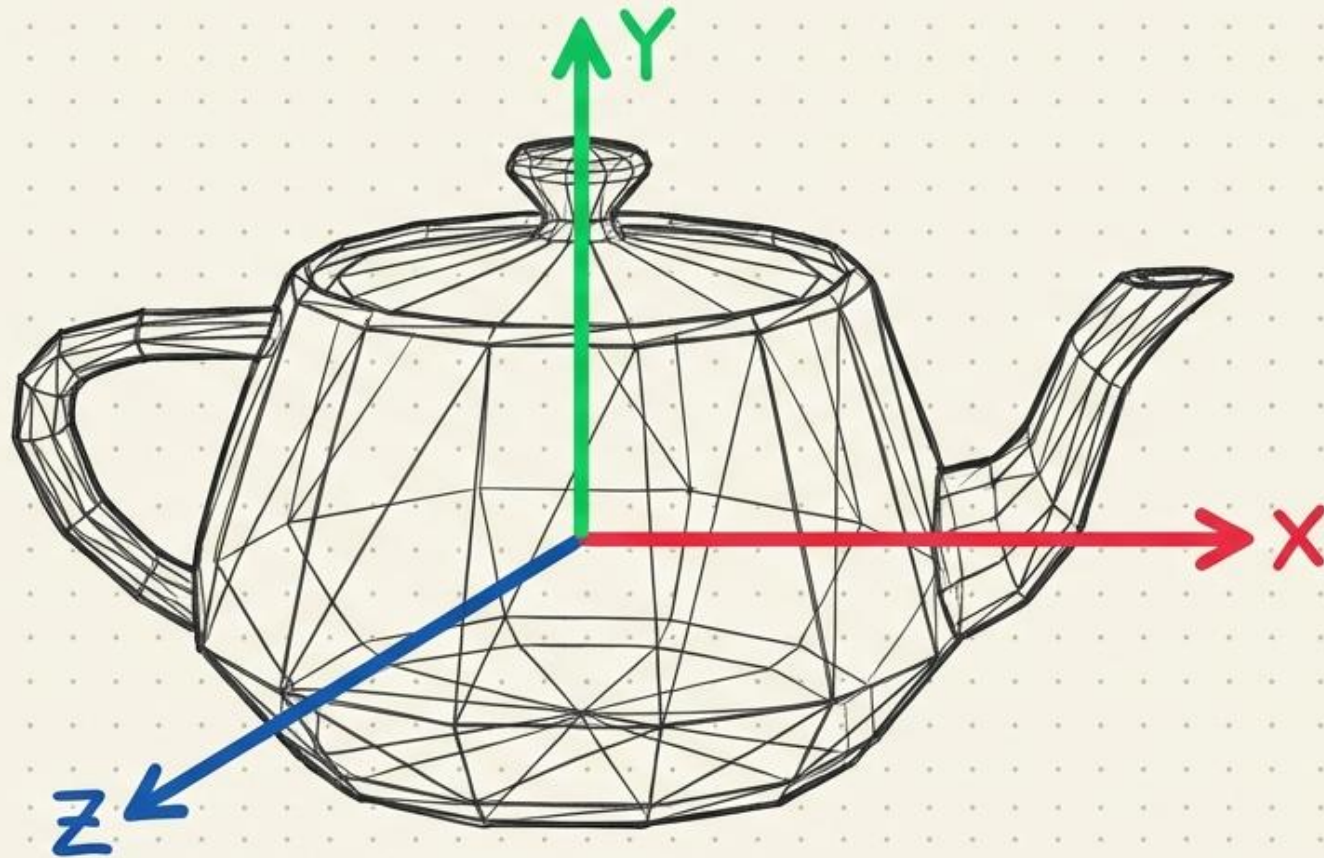


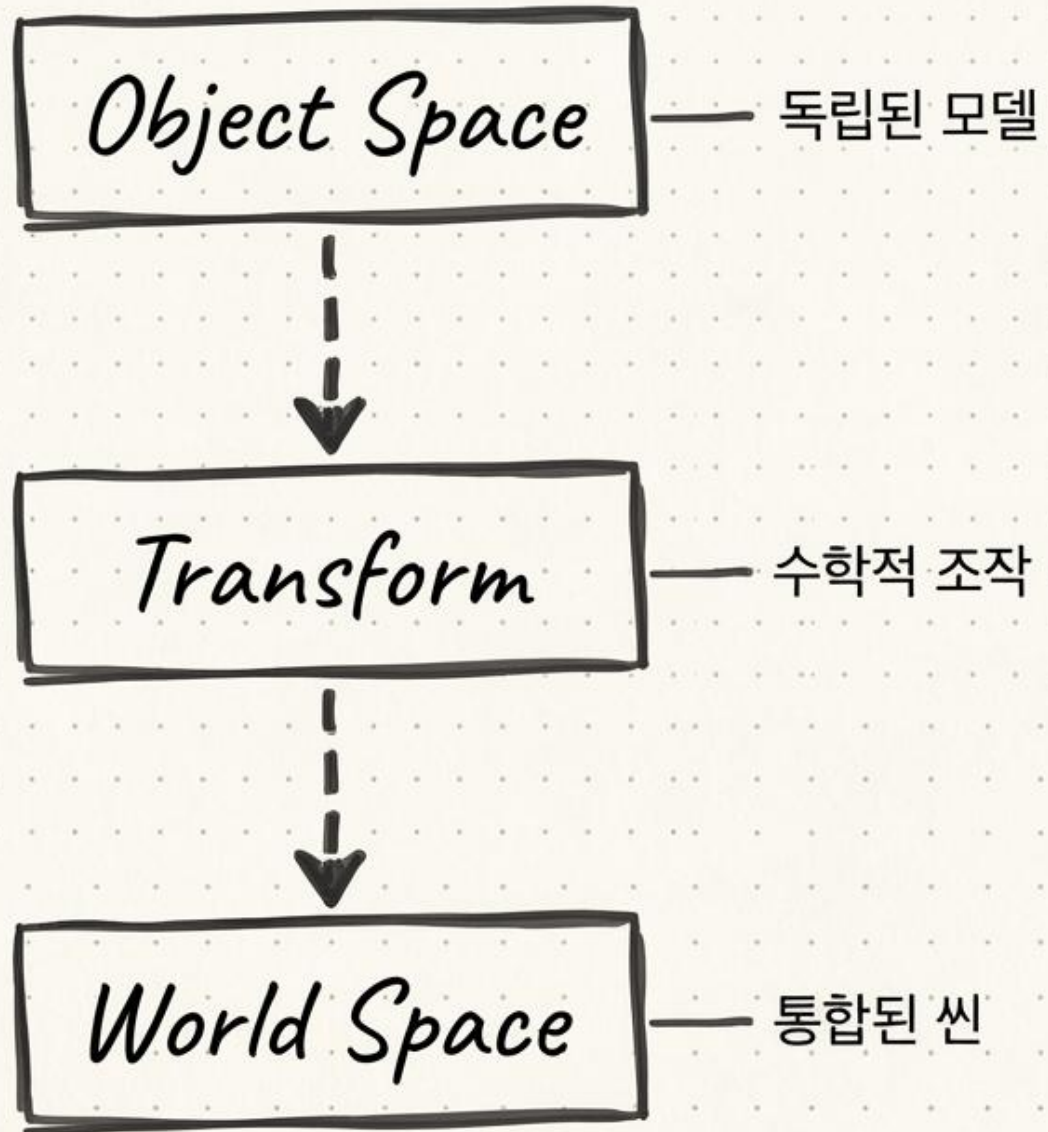
A SIMPLE PLANE, MADE OF CUBES



World Transform / Model Transform

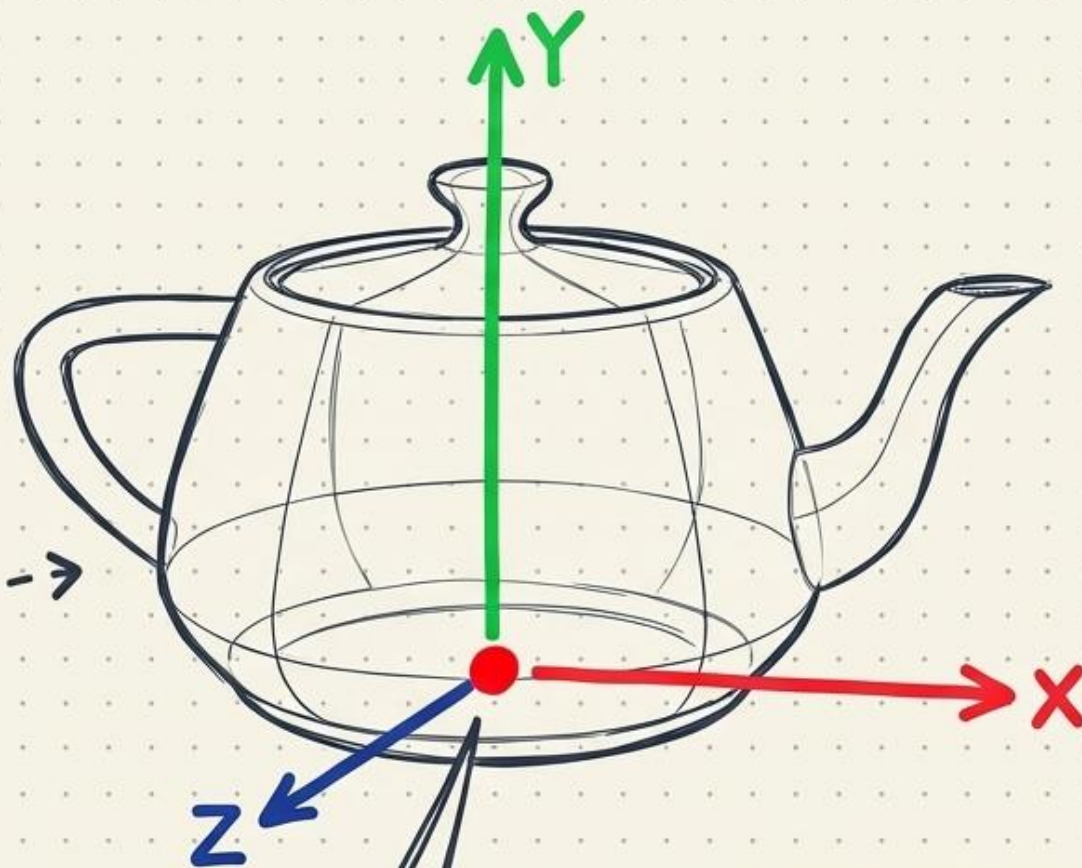
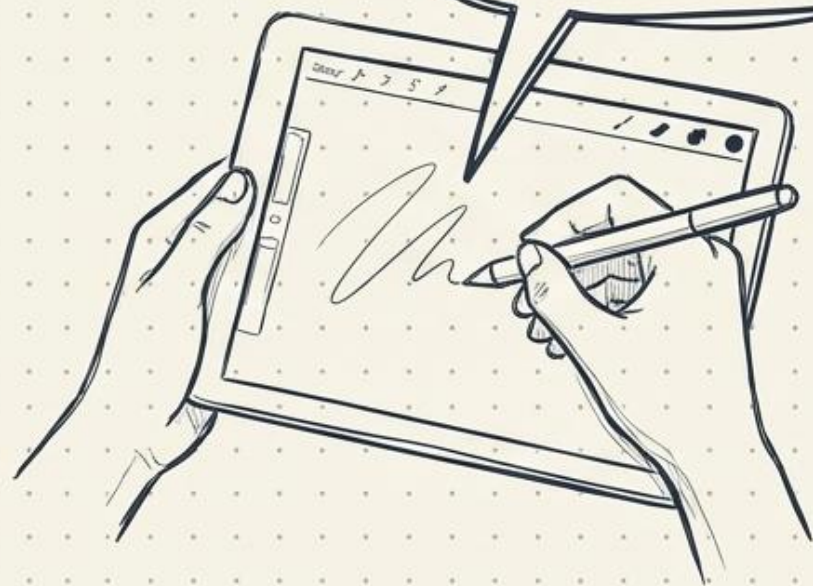
간략한 개요 및 핵심 용어 설명





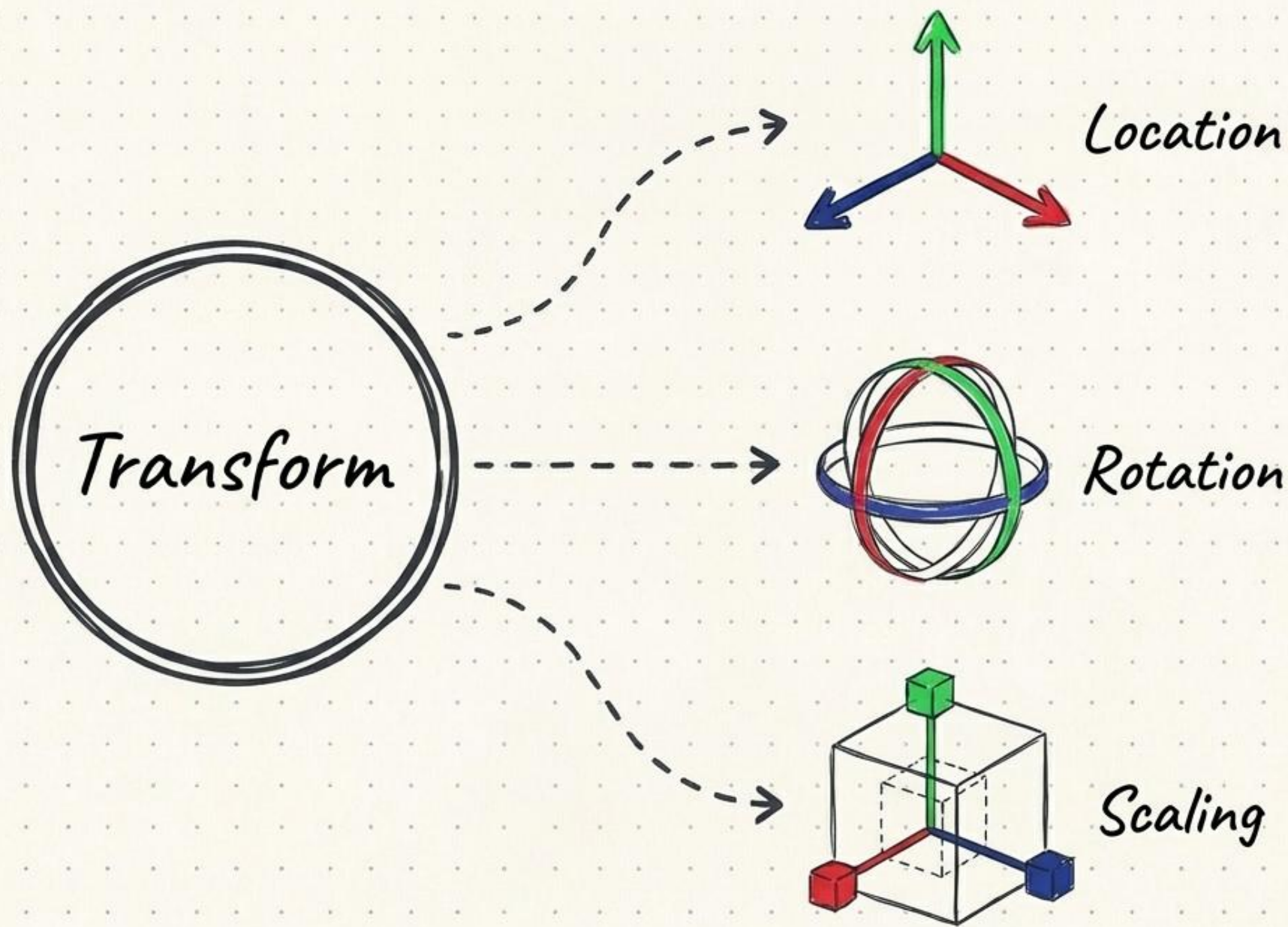
Object Space (Local Space)

디자이너가 객체를
처음 만들 때 사용하는
독립적인 좌표계.



기준: 모델의 중심 또는
바닥이 원점 (0, 0, 0).

비유: 나만의
고립된 작업 공간

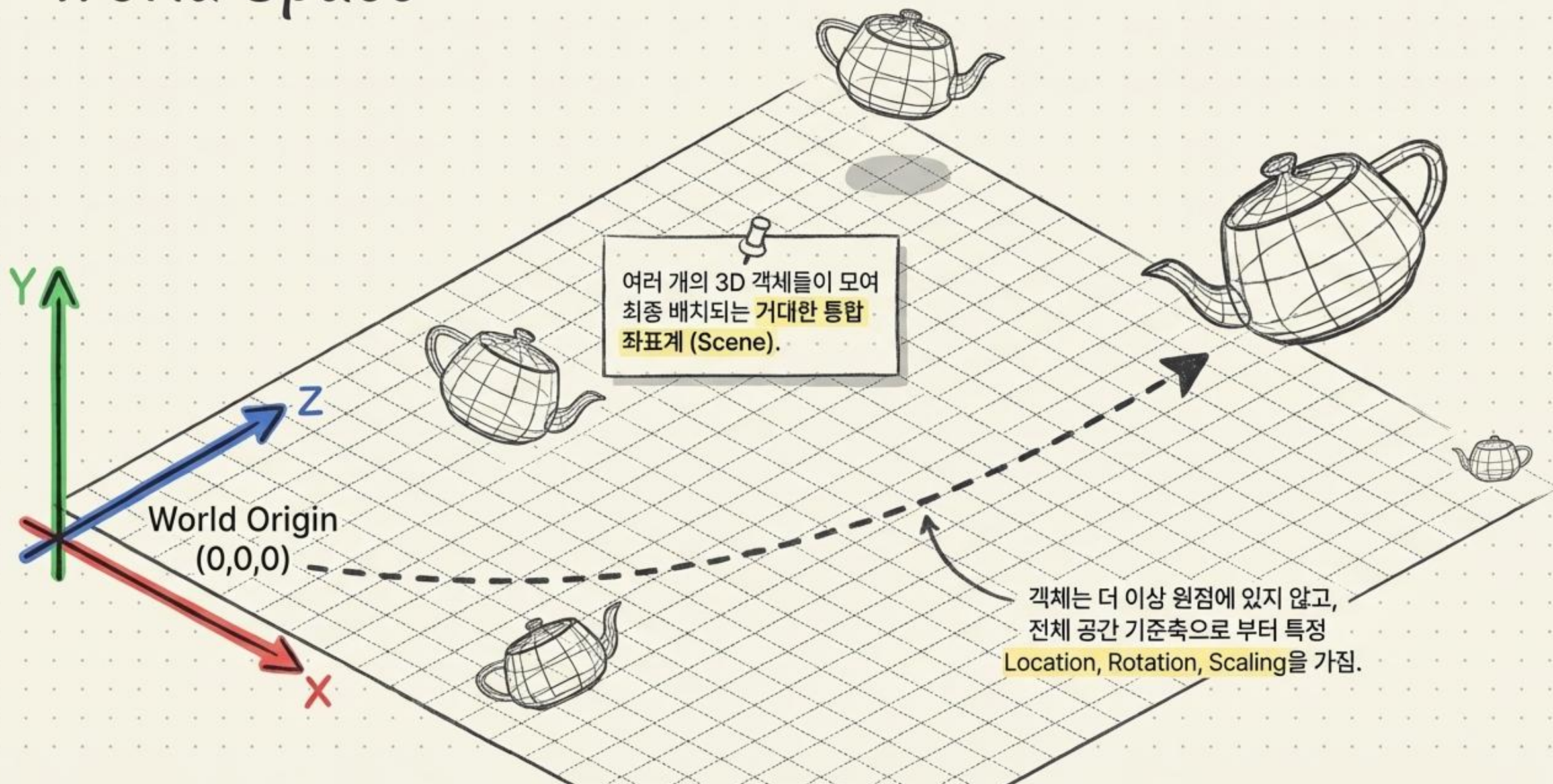


물체를 어디에
둘 것인가?
(위치 이동)

어느 방향으로
바라보게
돌릴 것인가?
(회전)

크기를 얼마나
키우거나
줄일 것인가?
(크기 조절)

World Space



Object Space

World Space

(기준 - Origin)

원점 = 모델의 중심

원점 = 전체 씬(Scene)의 중심

(상태 - State)

모델링 중인 고립 상태

변환(Transform)이 완료된
배치 상태

(비유 - Analogy)

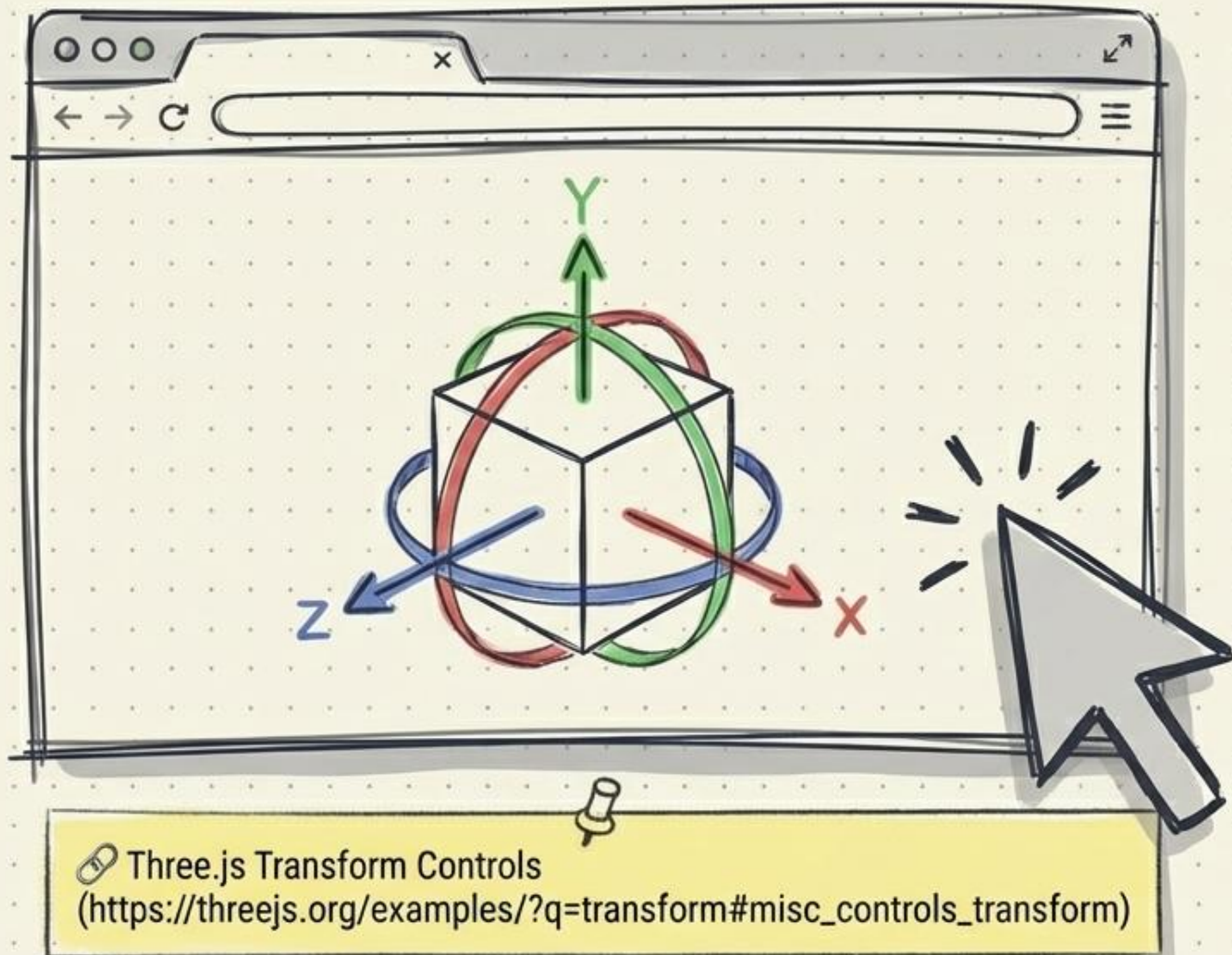
디자이너의 태블릿 스케치 보드

완성된 3D 디오라마 무대

Hands-on Demo

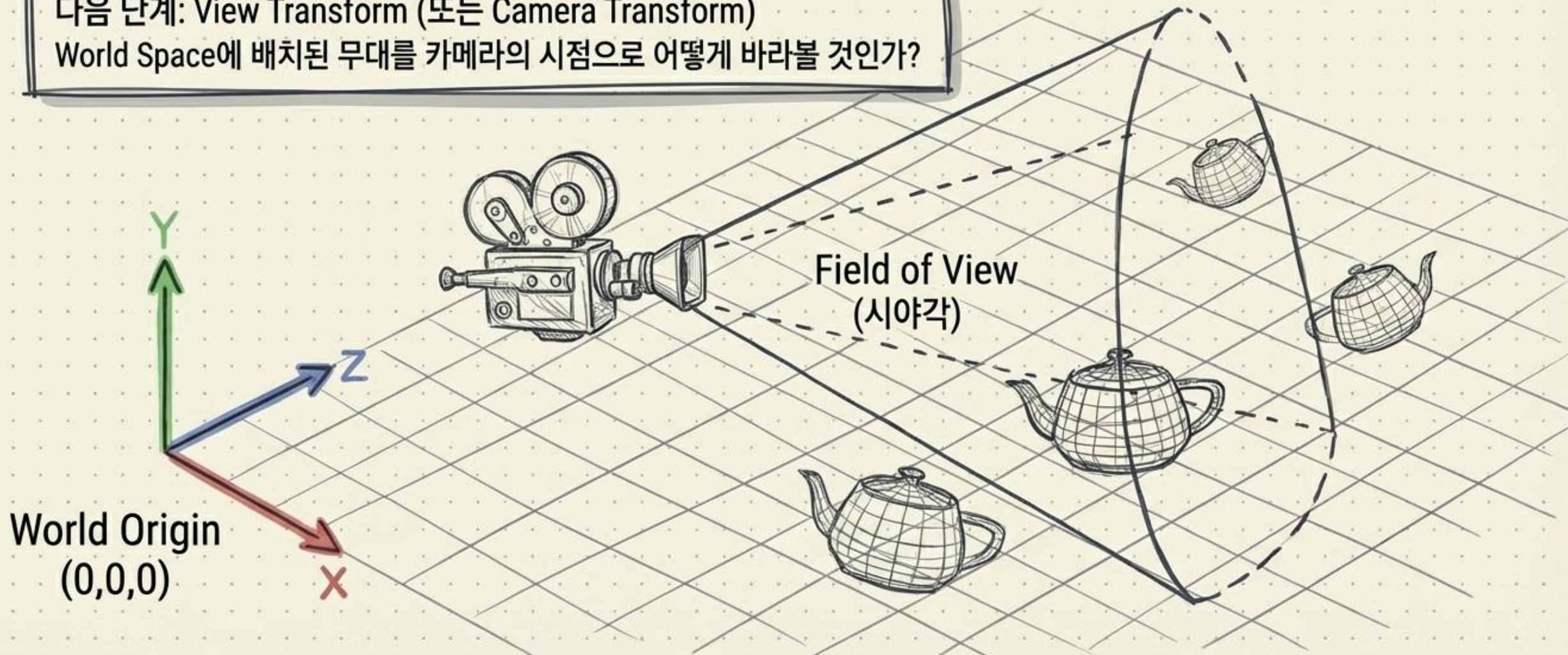
직접 Transform을 체험해 보세요.

Gizmo를 이용해 물체를 직접 조작하며
Location, Rotation, Scaling의
개념을 눈으로 확인하세요.

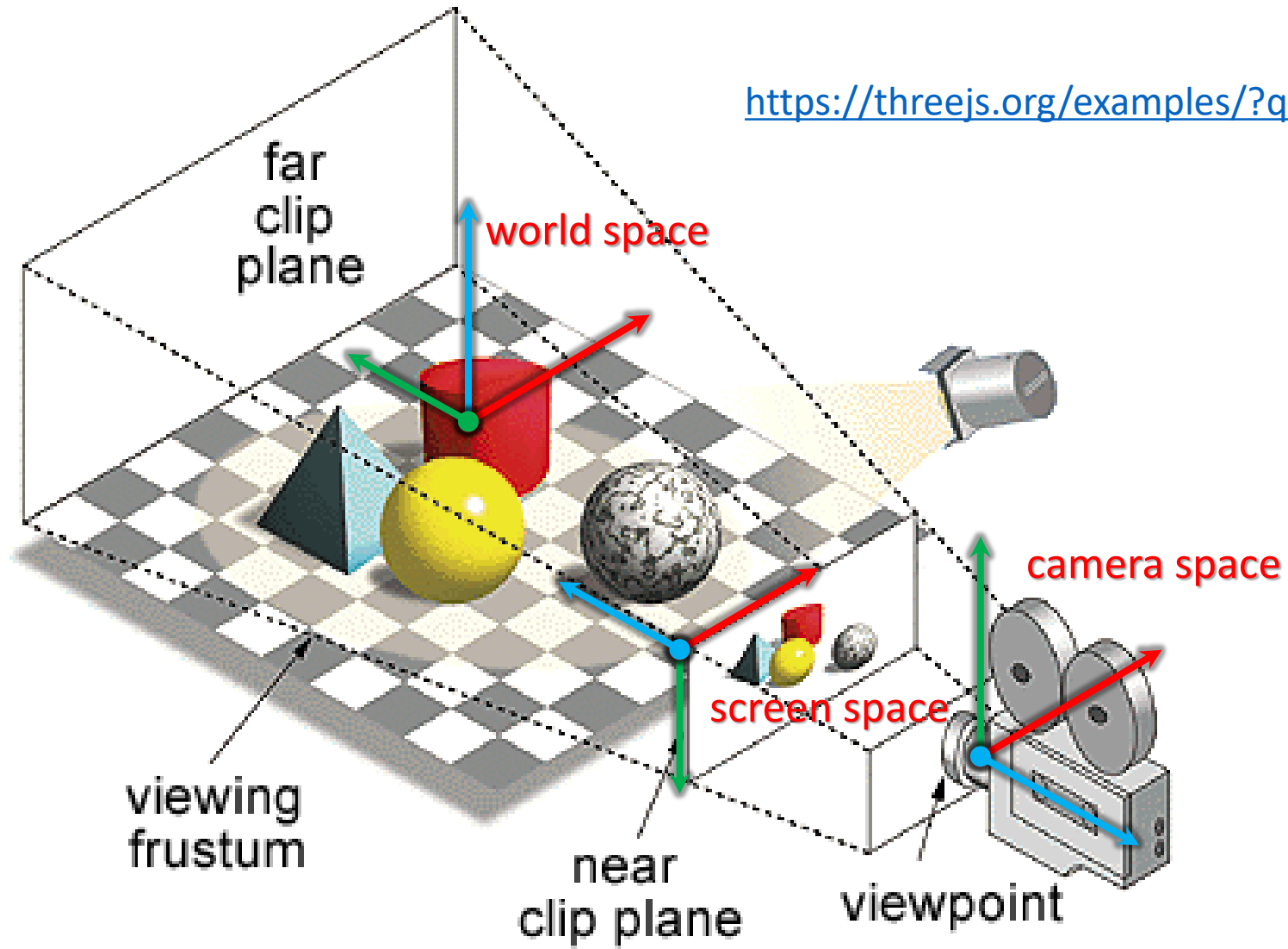


What's Next?

다음 단계: View Transform (또는 Camera Transform)
World Space에 배치된 무대를 카메라의 시점으로 어떻게 바라볼 것인가?



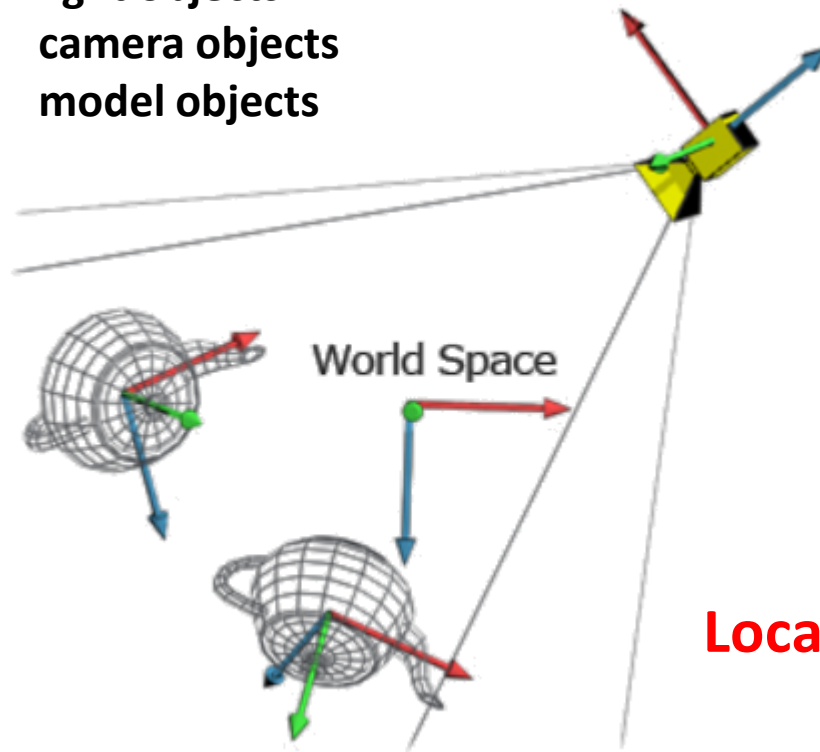
https://threejs.org/examples/?q=camera#webgl_camera



View Transform

We call this **View Transform**

light objects
camera objects
model objects

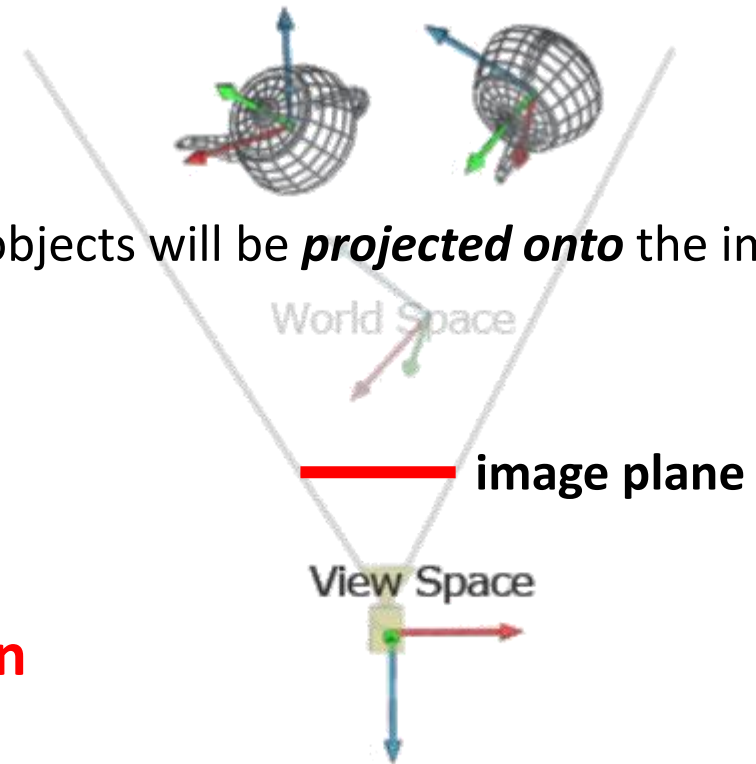


Location and Rotation

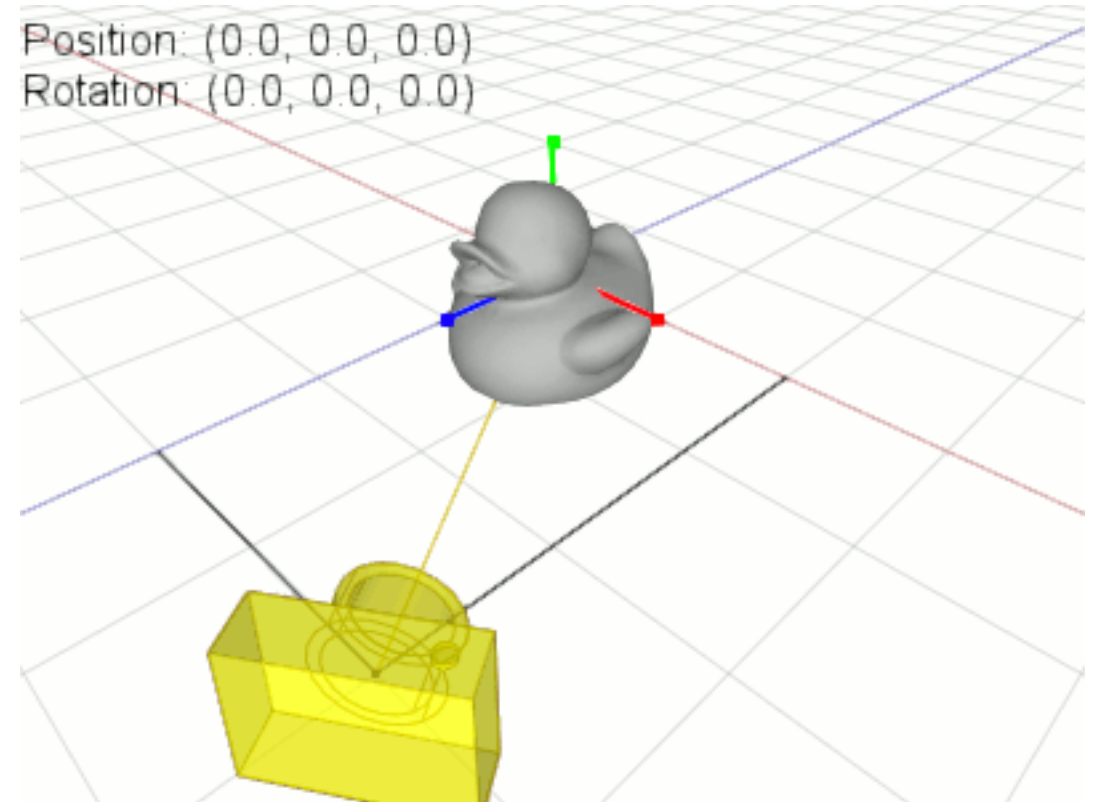
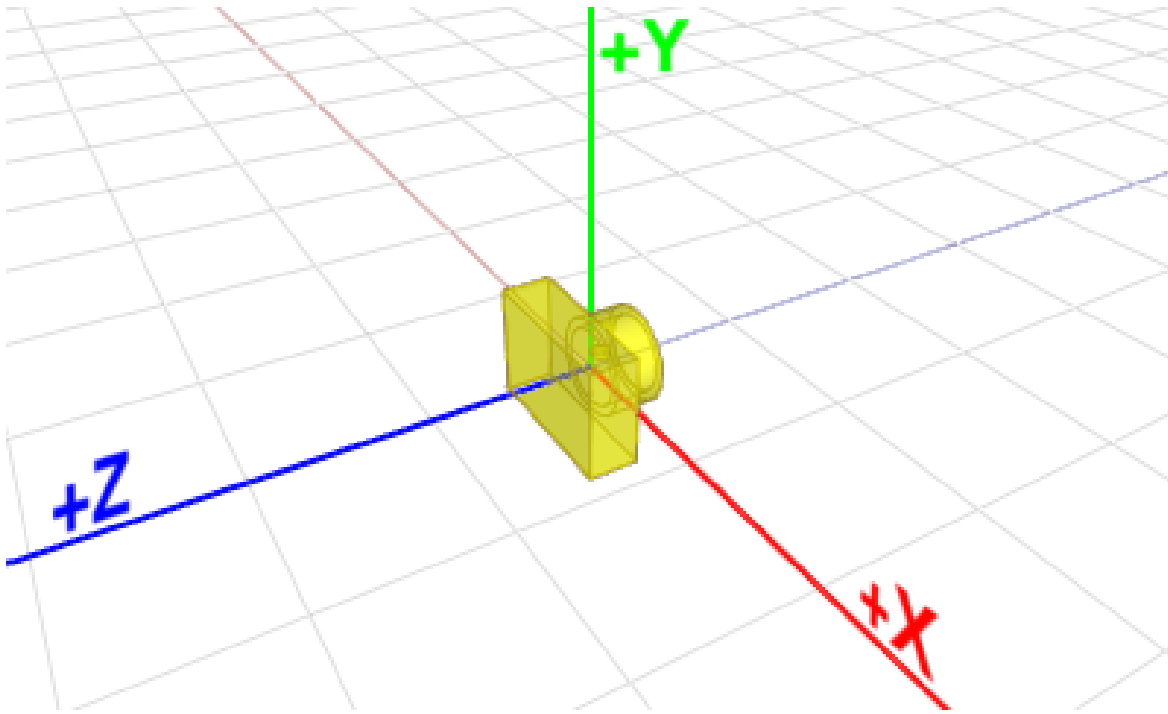
Camera Space (View Space)

- camera-origin & viewing-z coordinates
- unit is same as the unit in WS

the objects will be **projected onto** the image plane

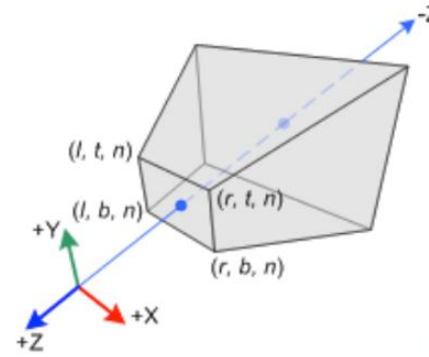
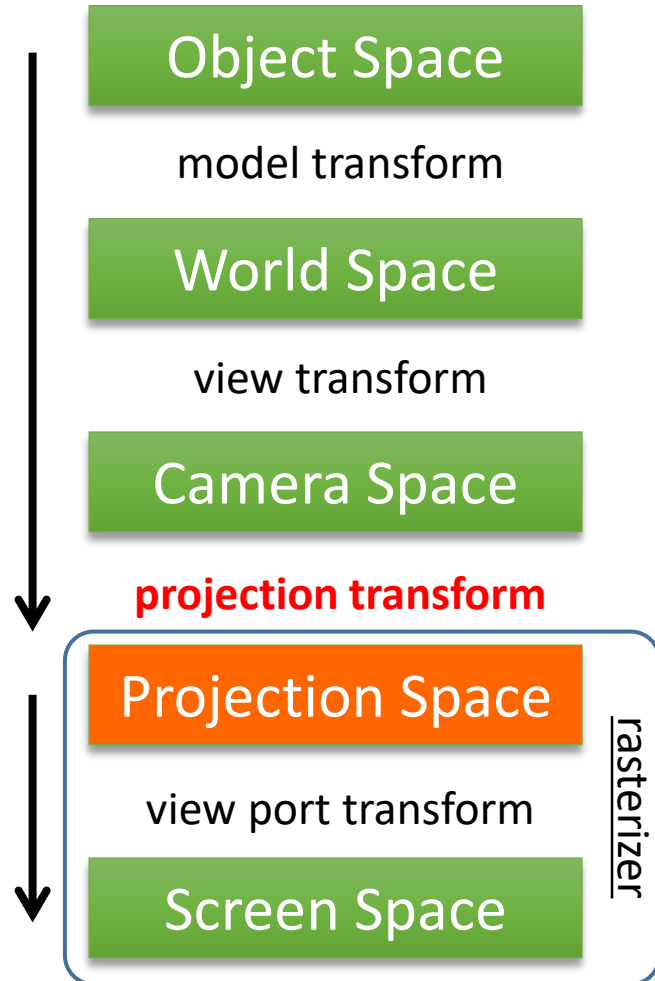


View Transform

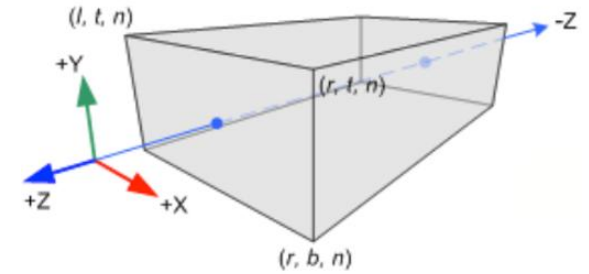


Transforms w.r.t. World Axes

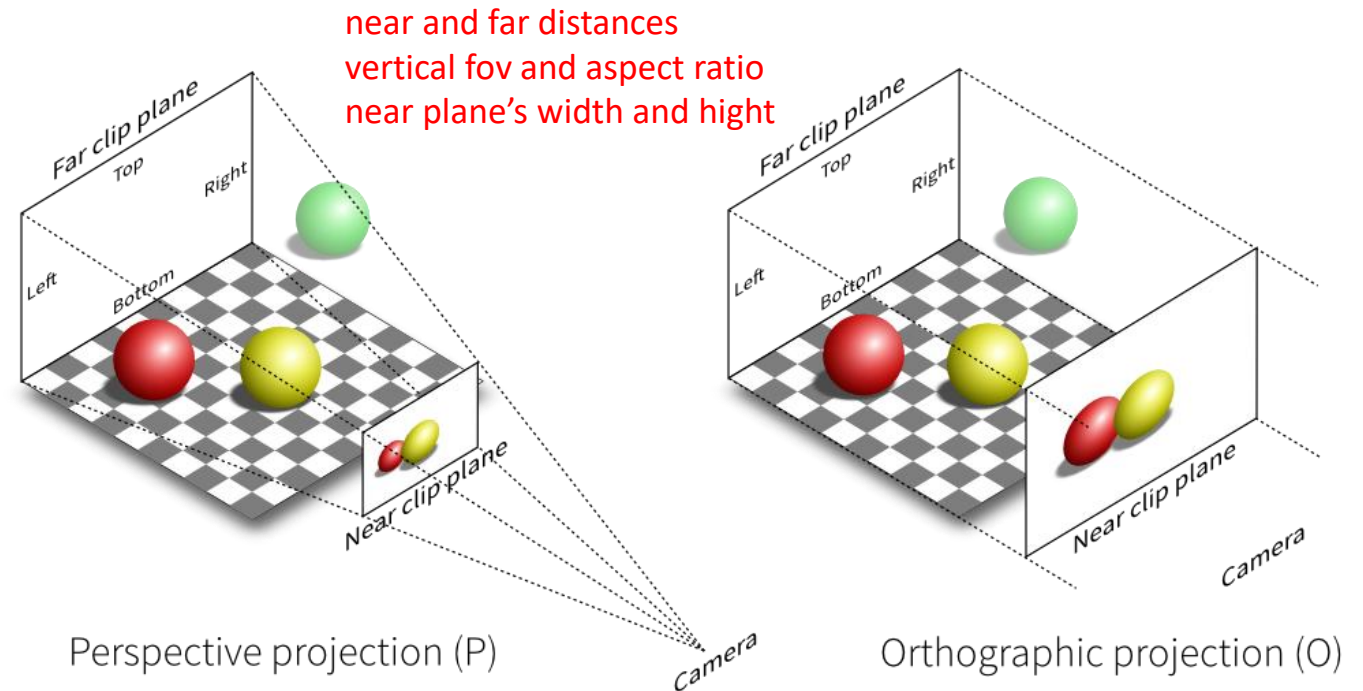
Projection Transform



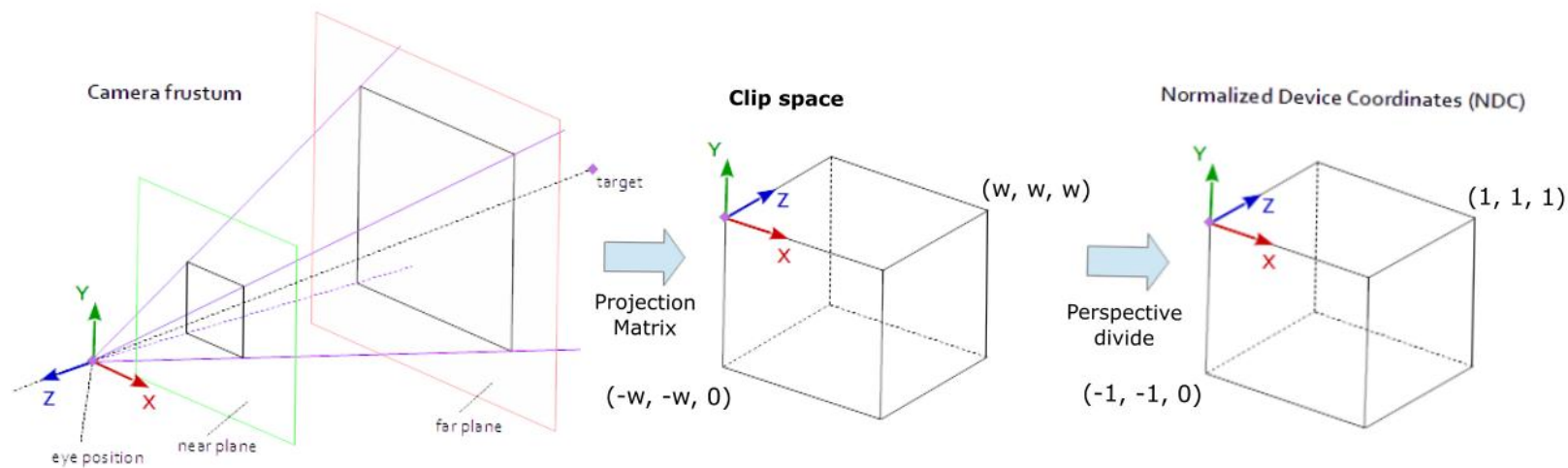
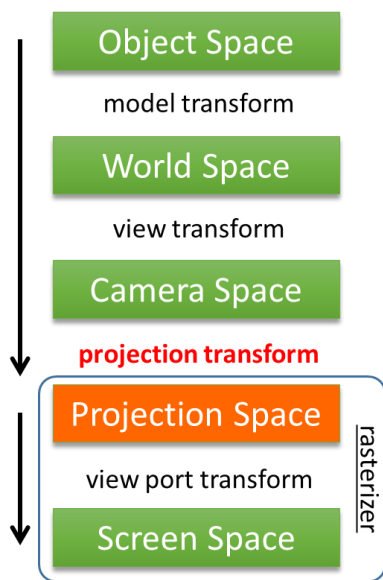
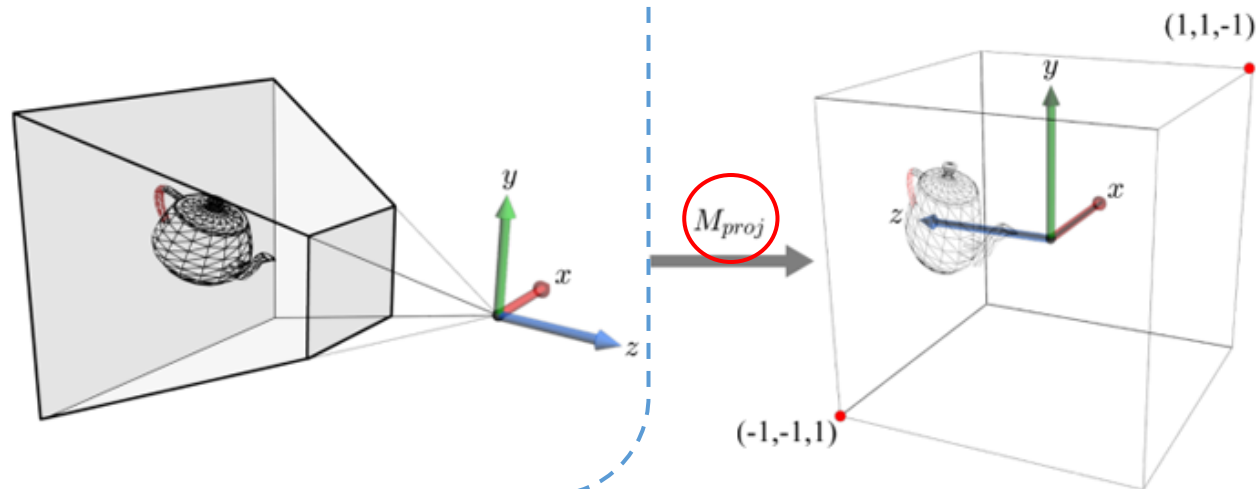
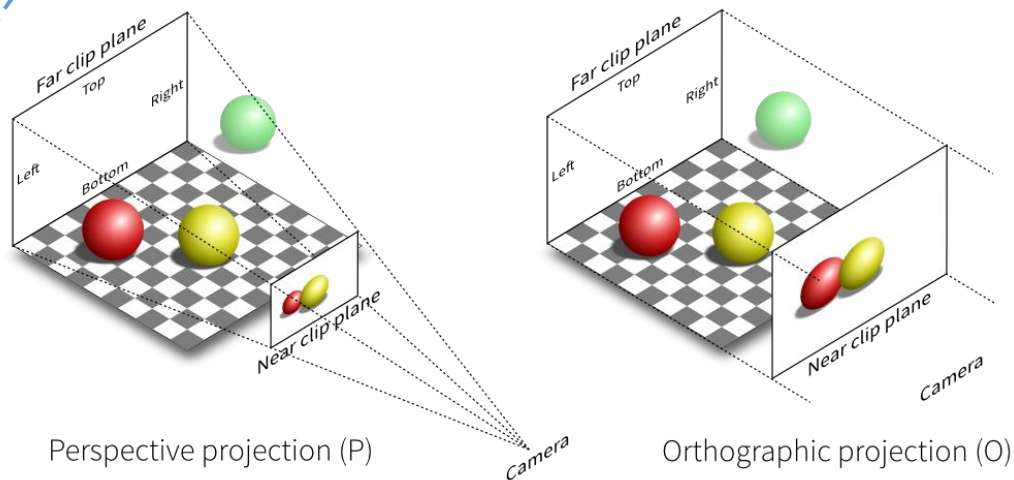
Perspective **view frustum**



Orthogonal **view frustum**



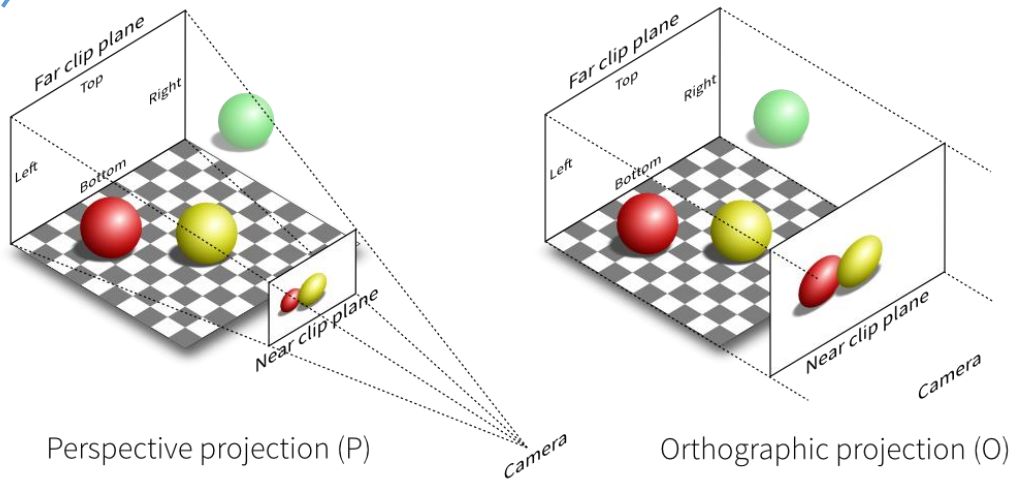
Projection Transform



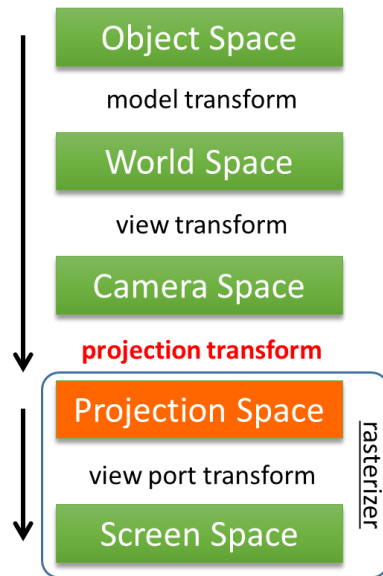
Projection Transform

Direct3D: (1, 1, 0)

OpenGL: (1, 1, -1)



- The view frustum is transformed into a cube shape
- Handled in Left-handed System (LHS)
(note that $z_n - z_f$ is always negative)
- Frustum culling (clipping) is performed in this space (very easy)

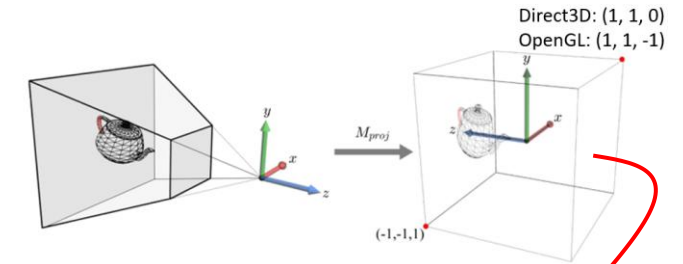
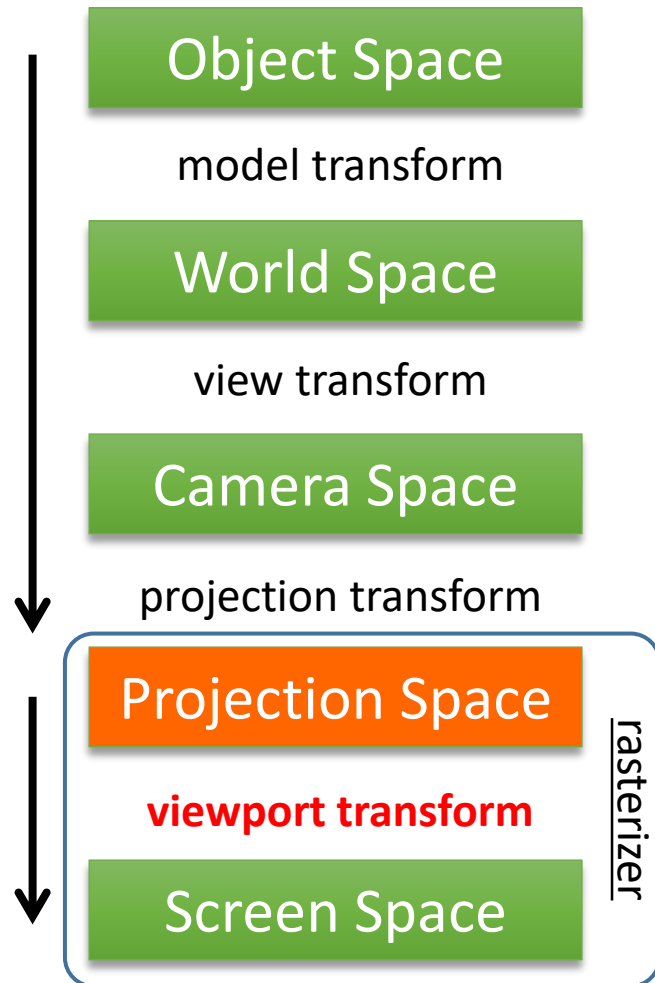


$$M_{proj} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & \frac{d_f * z_f - d_n * z_n}{z_n - z_f} & \frac{(d_f - d_n)(z_n * z_f)}{z_n - z_f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

$d_f = 1$ (both Direct3D and OpenGL)
 $d_n = 0$ (Direct3D), c.f., -1 (OpenGL)

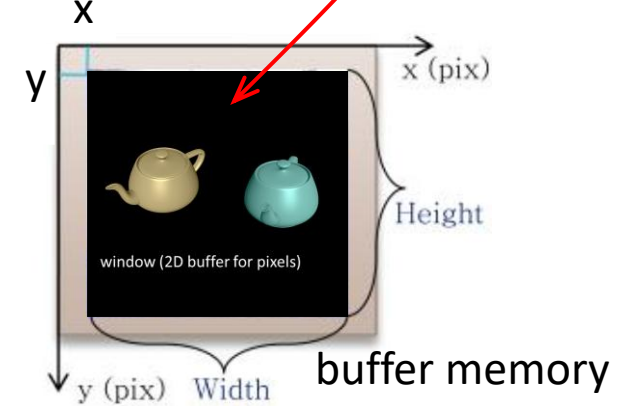
- The view frustum is transformed into a cube shape
- Handled in Left-handed System (LHS)
(note that $z_n - z_f$ is always negative)
- Frustum culling (clipping) is performed in this space (very easy)

Viewport Transform



$$\begin{pmatrix} x_w \\ y_w \\ z_w \end{pmatrix} = \begin{pmatrix} \frac{w}{2}x + \left(x + \frac{w}{2}\right) \\ \frac{h}{2}y + \left(y + \frac{h}{2}\right) \\ \frac{f-n}{2}z + \frac{(f+n)}{2} \end{pmatrix}$$

return to Right-handed System (RHS)



$$\begin{cases} -1 & \rightarrow x \\ 1 & \rightarrow x + w \end{cases}, \quad \begin{cases} -1 & \rightarrow y \\ 1 & \rightarrow y + h \end{cases}, \quad \begin{cases} -1 & \rightarrow n \\ 1 & \rightarrow f \end{cases}$$

(OpenGL)

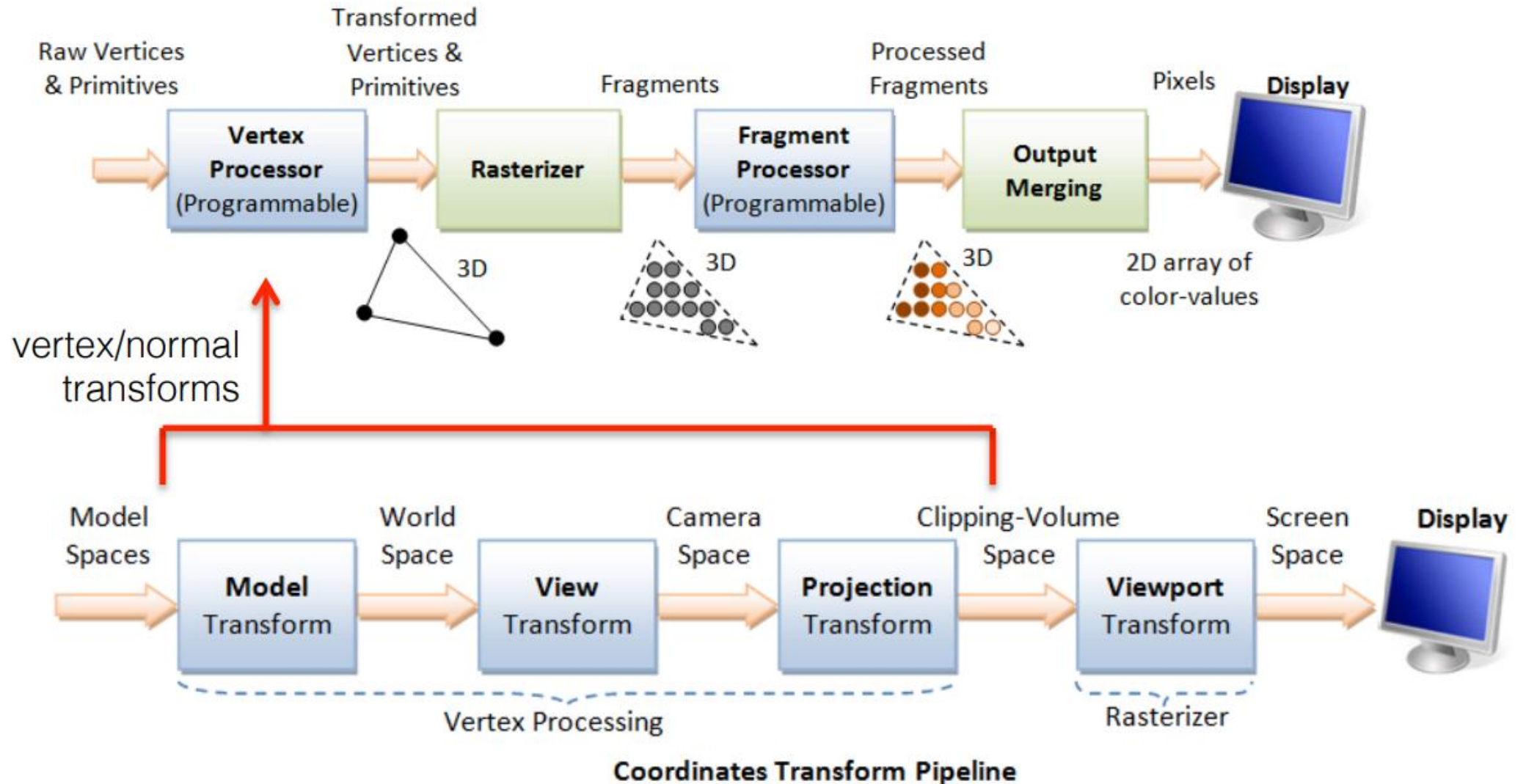
(DirectX)

$z_w = z$

What is rasterizer

<https://www.youtube.com/watch?v=t7Ztio8cwqM>

Where We Are in Graphics Pipeline



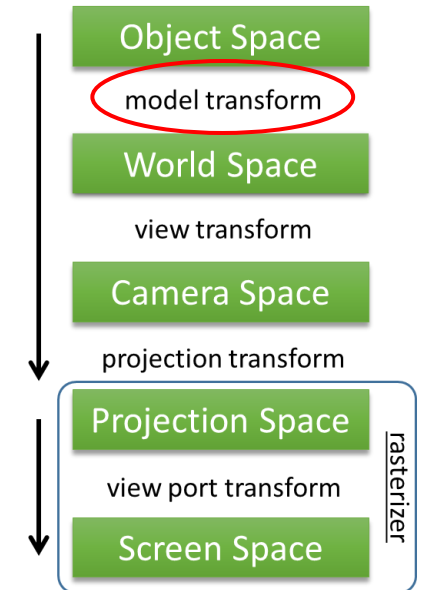


<https://www.youtube.com/watch?v=C8YtdC8mxTU&t=158s>

Scene Tree: Local and World Transforms

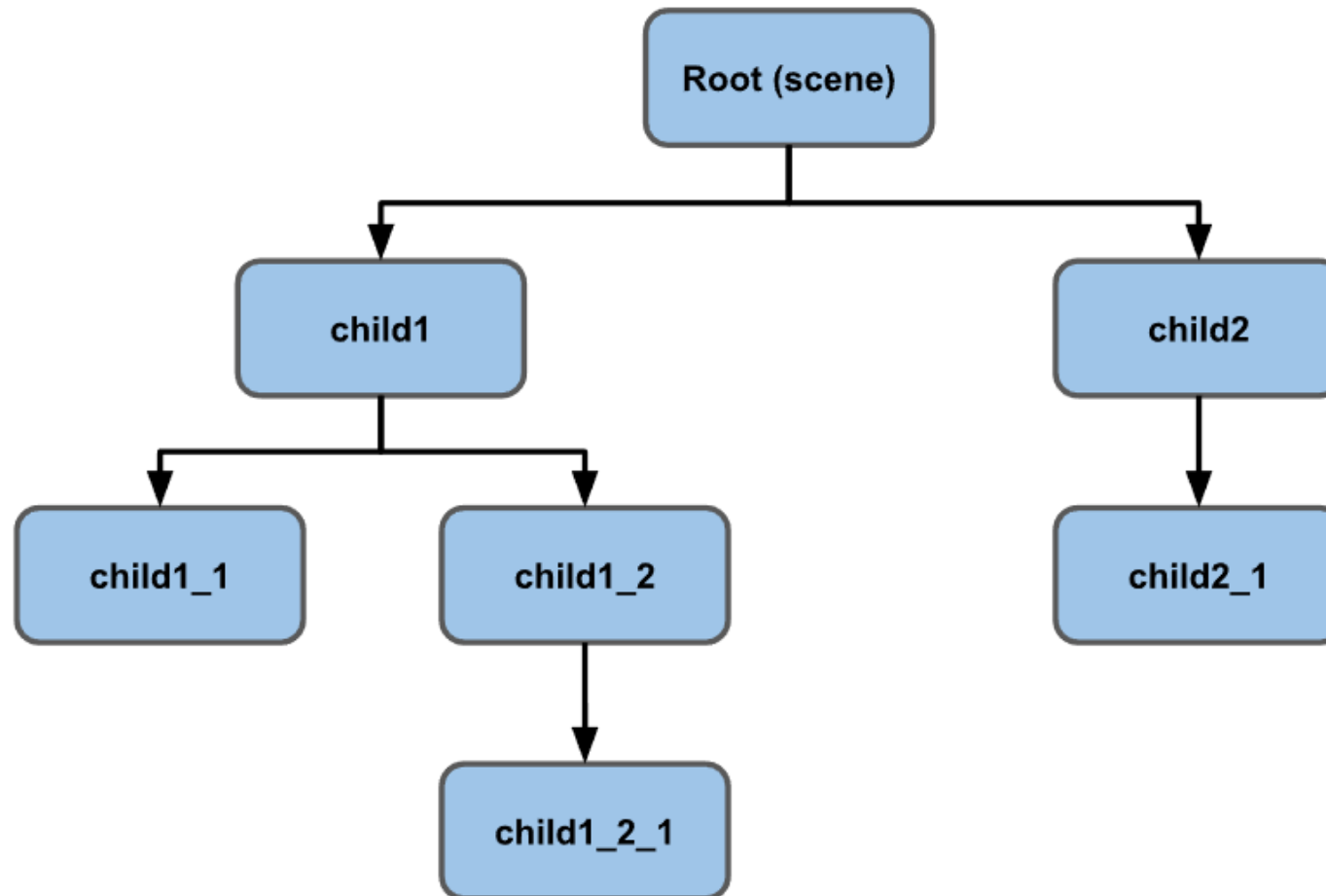
World (Model) Transform

- Most transform is handled as a model transform
- Scaling transform
 - When the unit of OS and WS is different
 - When the scene requires the resized model
- Translation
 - Just move the model in WS
- Rotation
 - Orientation setting
- The combination of these transforms is defined as **affine transforms!**



Local Transform vs. World Transform

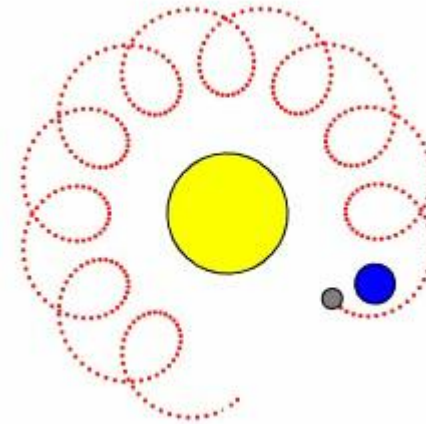
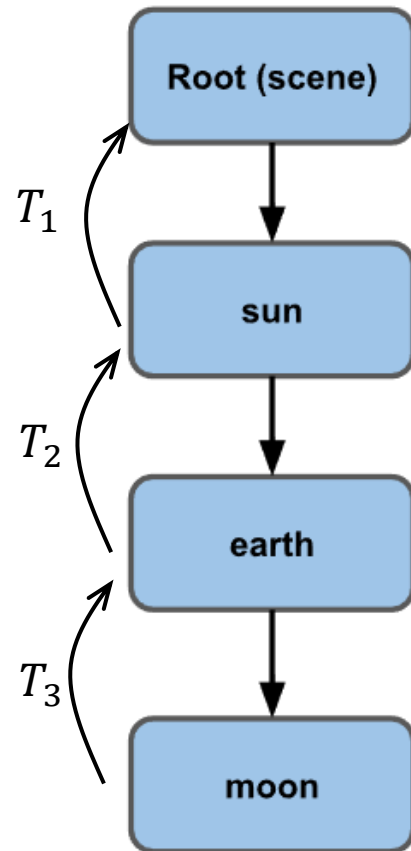
- Model transforms are highly related to the 3D scene tree structure (i.e., Scene Graph)
 - A tree structure contains '**hierarchical**' nodes (parent and children)



Scene graph

Local Transform vs. World Transform

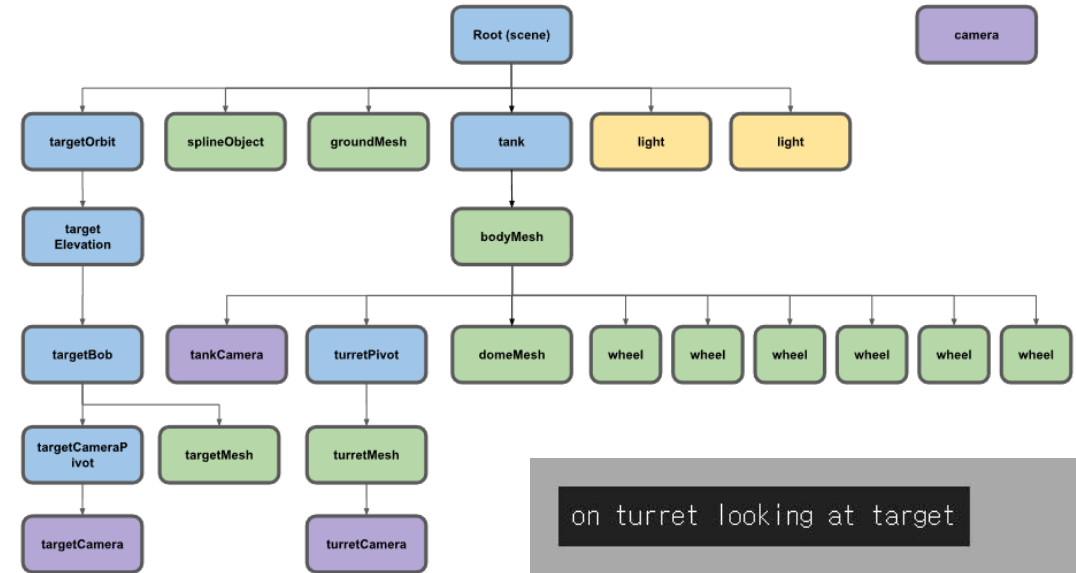
- Transform to the root (scene or world) is called **World Transform (or Model Transform)**
- Transform to the parent is called **Local Transform**



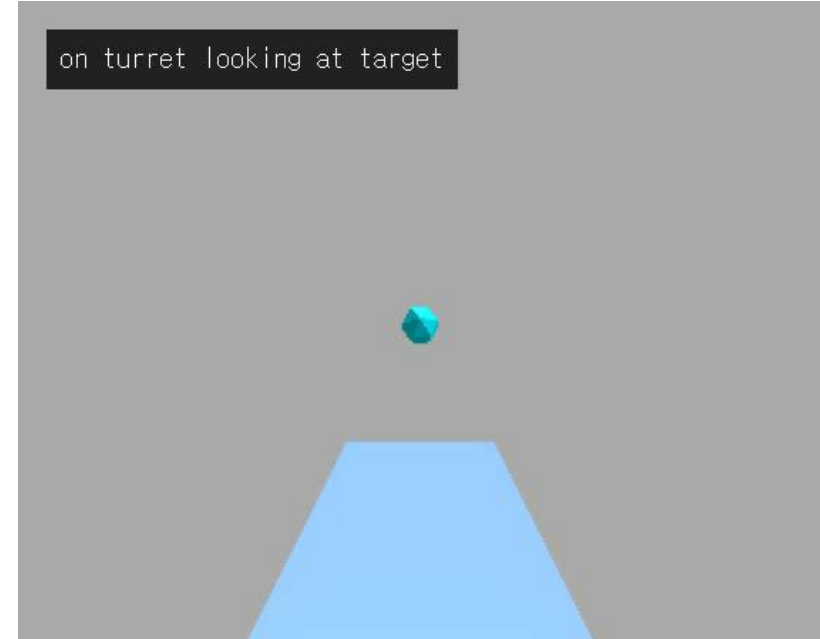
Question ☺

What is earth's local transform to the sun?
What is Moon's world transform?

Complex Scene Examples



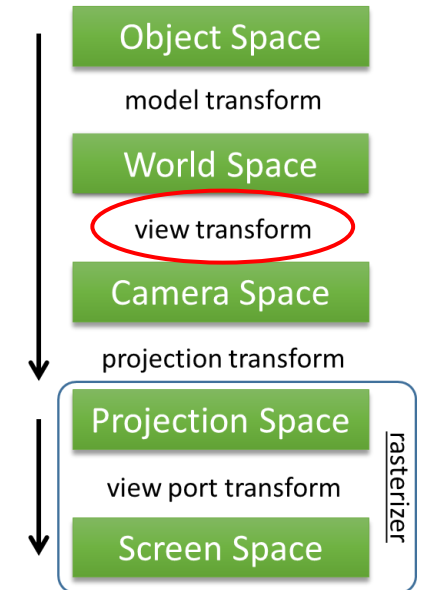
on turret looking at target



LookAt: View Transform

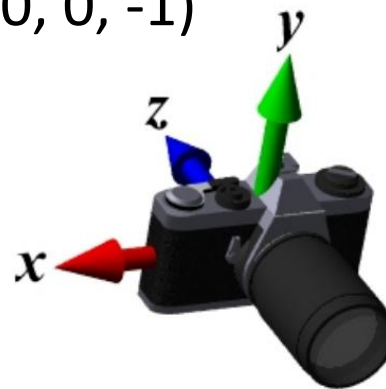
Revisit: View Transform

- No scaling (WS and CS use the same unit)
- Translation and Rotation (i.e., affine transformation)
- Simply determined by '*look-at*' context
 - Camera position (origin point)
 - Look-at position (reference point) or viewing direction (vector)
 - Up-vector (vector)
- The same technique for axis-alignment transform

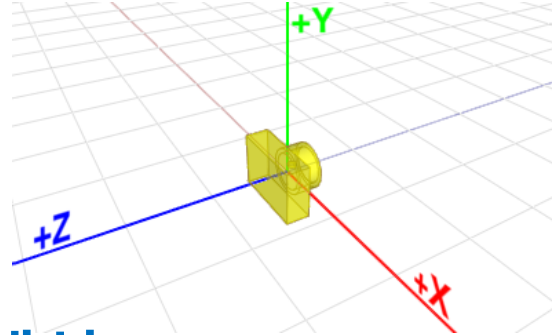


Revisit: Camera Space

- Coordinate system
 - Camera position at the origin
 - Z-axis aligned with the opposite of the viewing direction
 - Y-axis aligned with the up direction
- Computing the ***view transform*** leads to a problem if axis-alignment!!
 - Transform from World Space (WS) to Camera Space (CS)
 - Camera position to origin point (0, 0, 0)
 - Camera up to y-axis (0, 1, 0)
 - Camera view direction to minus z-axis (0, 0, -1)



Revisit: View Transform



world axes 관점에서 변환 해석

➔ **Extrinsic**

local (object or model) axes 관점에서 변환 해석

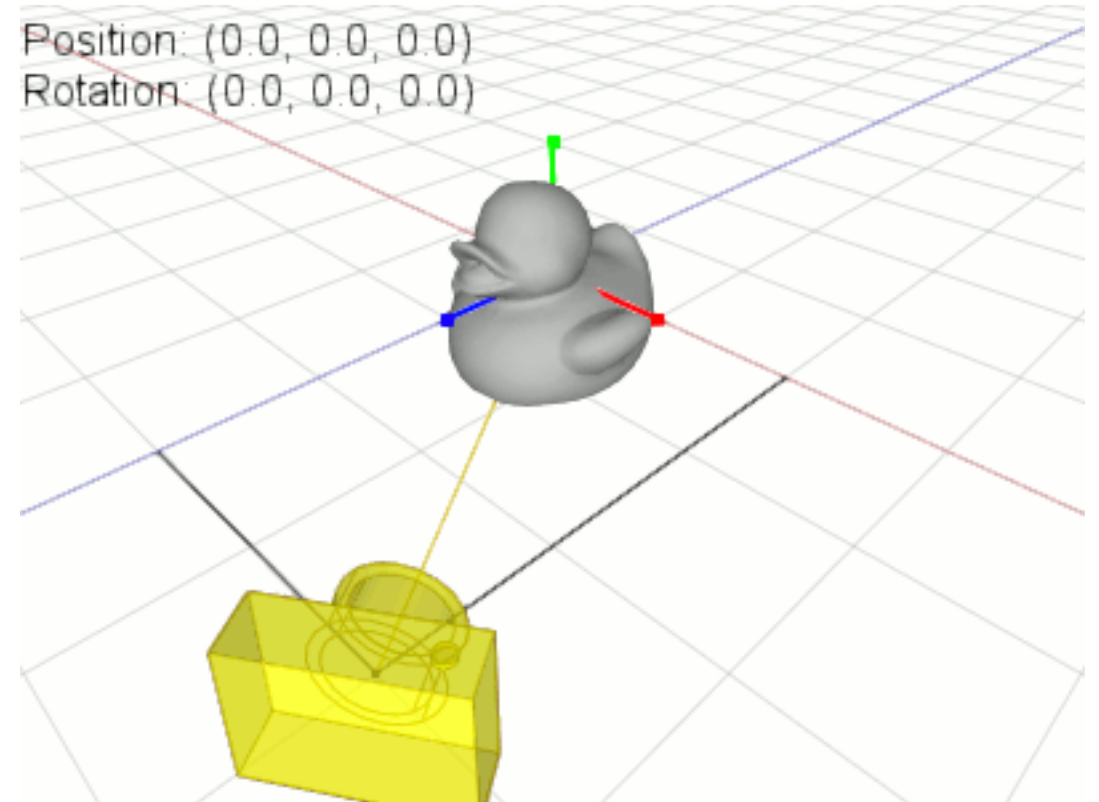
➔ **Intrinsic**

** 변환을 해석하는 순서!!

이수치는 Extrinsic ? Intrinsic ?

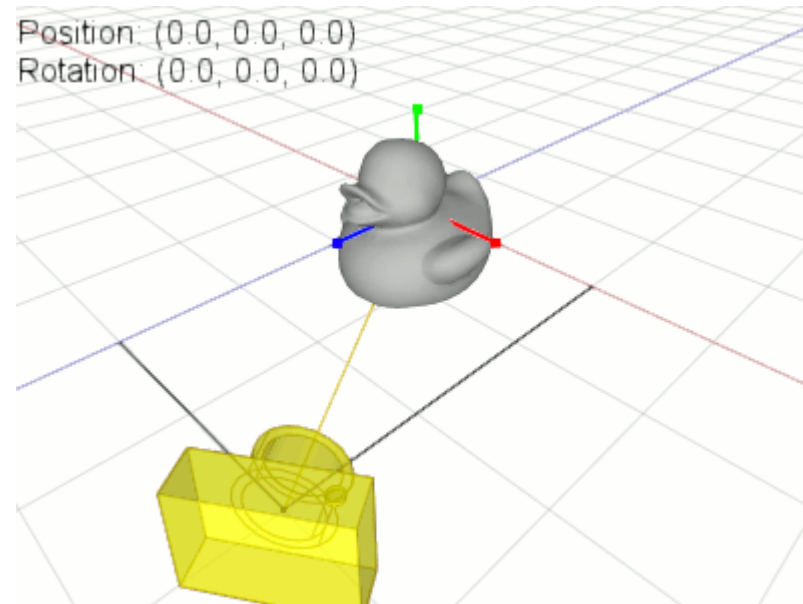
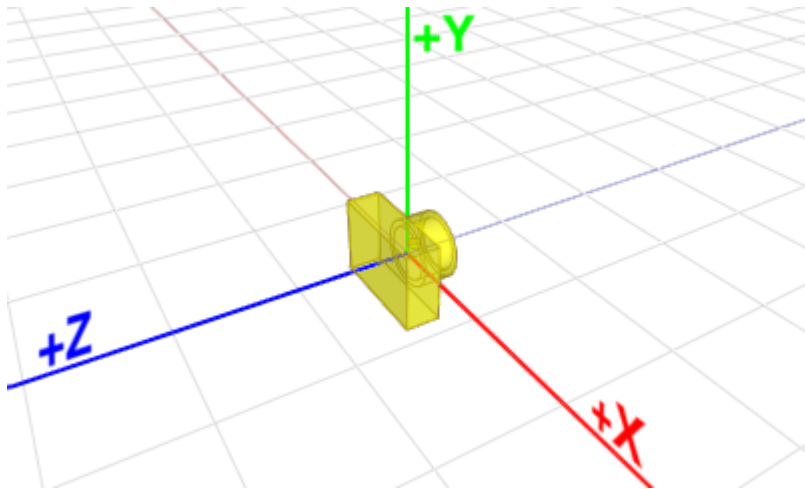
~~Position: (0.0, 0.0, 0.0)~~

~~Rotation: (0.0, 0.0, 0.0)~~



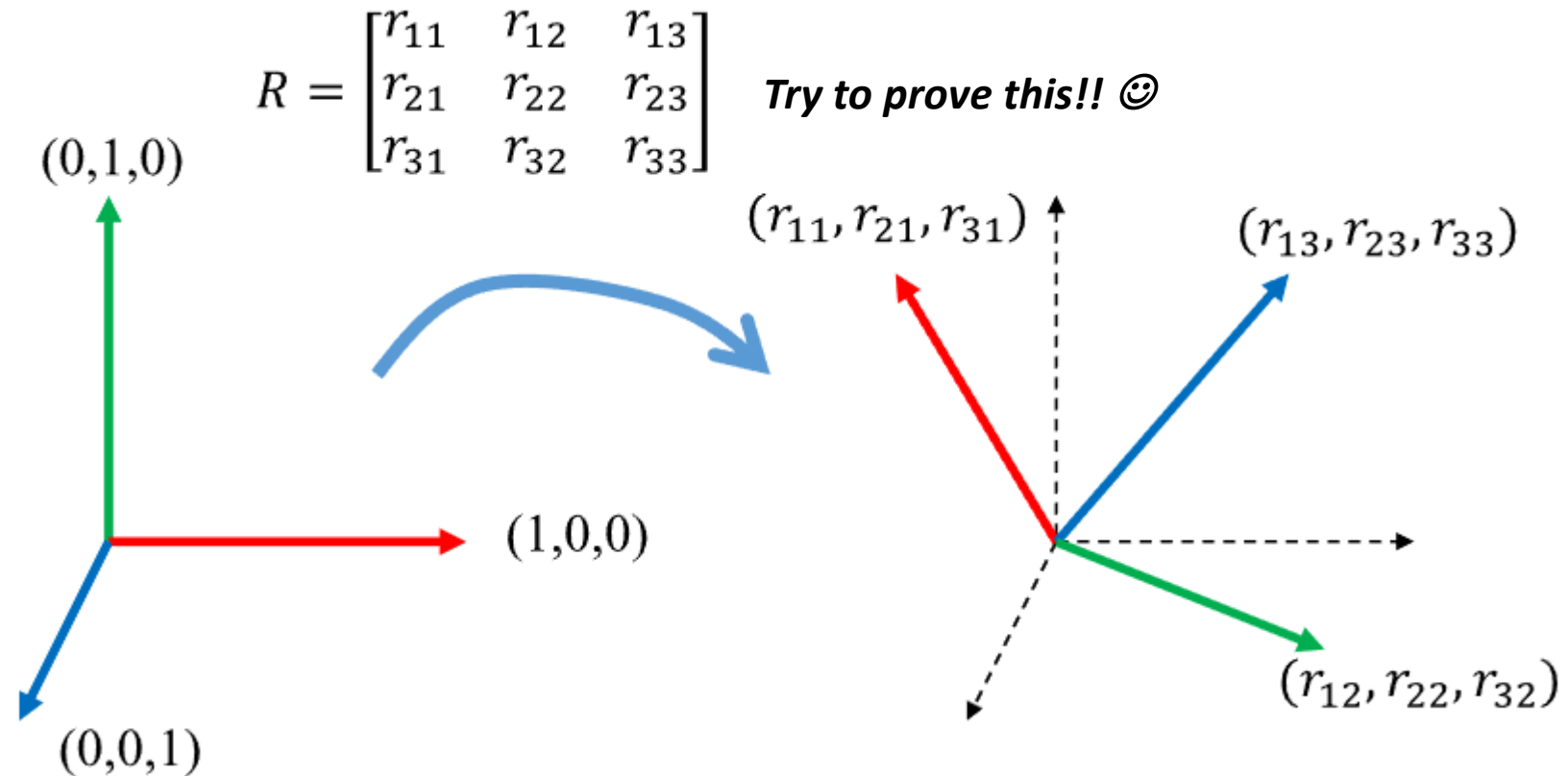
View Matrix Computation

- Method1) Consider the transform in the context of **EXTRINSIC** transform
 - First, translate to the camera position to WS origin $\rightarrow T$
 - Second, rotate to align the axes (camera axes to world axes) $\rightarrow R$
 - Then, the view transform is RT



Axis-alignment Problem

Rotation from Orientation A to Orientation B



Food for Thoughts:

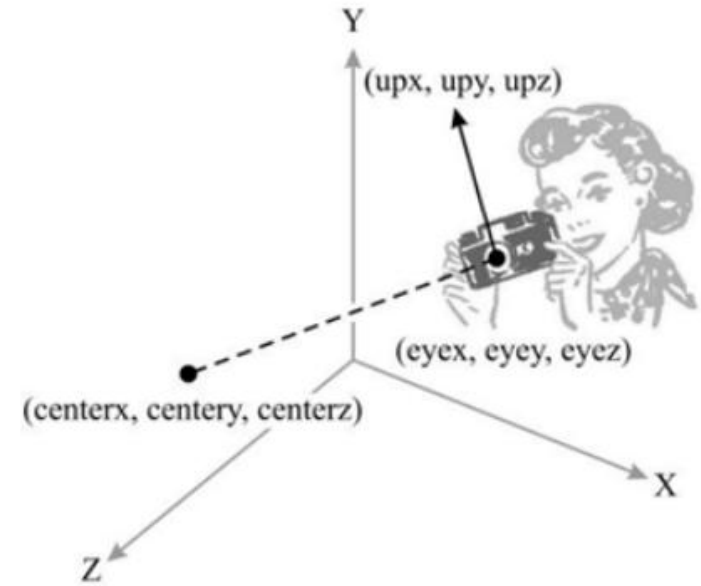
**Compute T from A space to B space
(given 3 axes and origin of each space)**

Use LookAt!

View Matrix Computation

specify camera by

- eye position $eye = \begin{pmatrix} eye_x \\ eye_y \\ eye_z \end{pmatrix}$
- reference position $center = \begin{pmatrix} center_x \\ center_y \\ center_z \end{pmatrix}$
- up vector $up = \begin{pmatrix} up_x \\ up_y \\ up_z \end{pmatrix}$



View Matrix Computation

specify camera by

- eye position $eye = \begin{pmatrix} eye_x \\ eye_y \\ eye_z \end{pmatrix}$
- reference position $center = \begin{pmatrix} center_x \\ center_y \\ center_z \end{pmatrix}$
- up vector $up = \begin{pmatrix} up_x \\ up_y \\ up_z \end{pmatrix}$

compute 3 vectors:

$$z^c = \frac{eye - center}{\|eye - center\|}$$

$$x^c = \frac{up \times z^c}{\|up \times z^c\|}$$

$$y^c = z^c \times x^c$$

View Matrix Computation

- Method2) Consider the transform in the context of **EXTRINSIC transform**
 - This leads to CS2WS transform (tip: think of CS as OS), or **world transform**
 - First, rotate to align the axes (world axes to camera axes) $\rightarrow R$
 - Second, translate the origin to the camera position $\rightarrow T$
 - Then, inverse transform of CS2WS (TR) is the view matrix ($R^{-1}T^{-1}$)
 - Note:
 - Rotation matrix is an orthogonal matrix, i.e., $R^T R = R R^T = I$
 - Meaning that $R^T = R^{-1}$
 - The inverse of the translation is the minus translation

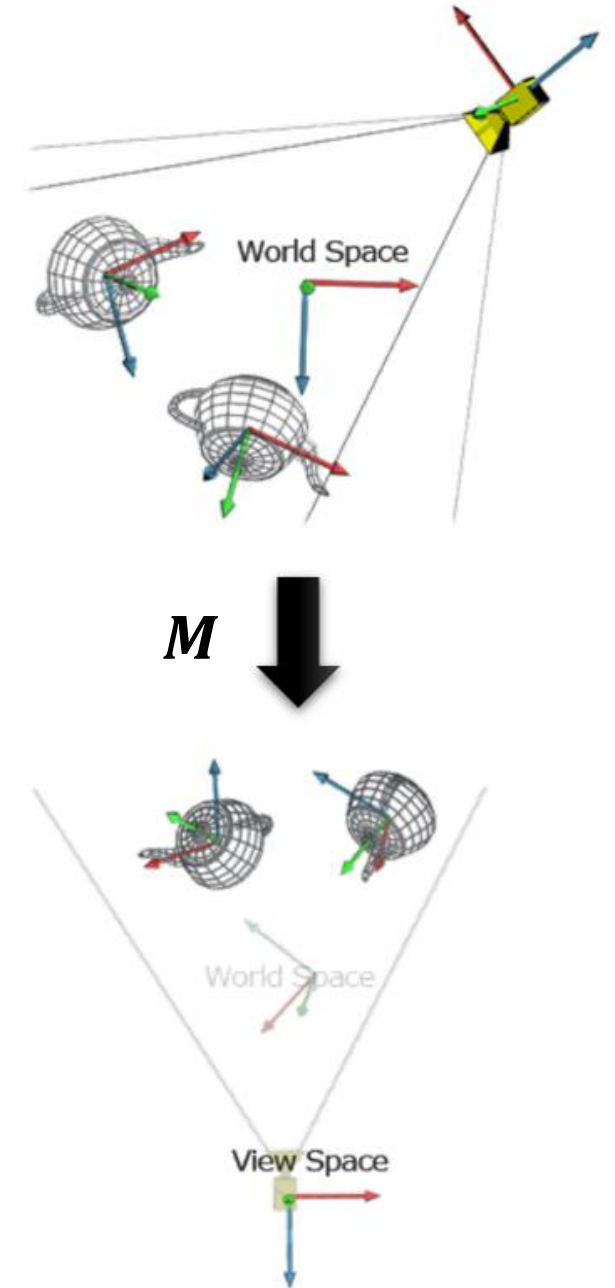
View Matrix Computation

$$z^c = \frac{eye - center}{\|eye - center\|}$$

$$x^c = \frac{up \times z^c}{\|up \times z^c\|}$$

$$y^c = z^c \times x^c$$

$$M = R \cdot T(-e) = \begin{pmatrix} x_x^c & x_y^c & x_z^c & -(x_x^c eye_x + x_y^c eye_y + x_z^c eye_z) \\ y_x^c & y_y^c & y_z^c & -(y_x^c eye_x + y_y^c eye_y + y_z^c eye_z) \\ z_x^c & z_y^c & z_z^c & -(z_x^c eye_x + z_y^c eye_y + z_z^c eye_z) \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



Practical Code: View Transform

- Look-at position (reference position) or view vector is obviously defined
 - Normalized view vector $\mathbf{v}$ is used for the computation
- How about the exact 'up' direction?
 - We will normally/easily take *rough* 'up' vector $\mathbf{u}_{tmp}$ (normalized)
 - Then, compute 'right' vector by cross-dot operation
 - ☺
 - Finally, compute the exact 'up' vector by cross-dot operation
 - ☺
- Almost of graphics math libraries provide the function that compute the view matrix from
 - Look-at position or view direction
 - Up vector
 - Camera position
 - e.g., glm::lookAtRH or Object3D.lookAt... (note: threejs computes just a rotation!)

Questions? 😊